

Bases de Datos Activas

Miguel Rodríguez Penabad

Laboratorio de Bases de Datos

Universidade da Coruña



UNIVERSIDADE DA CORUÑA

4 de marzo do 2024



Introducción

BD Pasivas vs. Activas

Bases de datos pasivas

- Son meros almacéns de datos
- Toda a xestión é explícita

Bases de datos activas

- Actividade básica: controles mínimos con restricións (clave primaria, foránea, ...).
- Máis avanzada: A BD “Reacciona” ante eventos (normalmente cambios nos datos)
- Utilidade (exemplos):
 - Control de restriccións
 - Reglas de negocio: Un empleado non pode ganar máis que o seu xefe
 - Validacións: NIF correcto, salario positivo, sexo é H ou M, ...
 - Actualización de datos
 - Xeneración de claves primarias
 - Tabla facturas con campo total autoactualizado
 - Accións externas á base de datos
 - Alerta de stock mínimo de produtos
- A actividade máis avanzada impleméntase normalmente con triggers (disparadores)

Introducción

Actividade sen triggers

Restriccións	Consideracións
• not null • unique e clave primaria • check • Clave foránea	• A acción sempre é abortar (excepto accións referenciais en FK) • check: só a nivel de fila (SXBDs non admiten subconsultas) • Non reacciona ante borrado de filas (excepto FK: reacciona ante borrados na táboa referenciada) • Ventaxa: Poden aplazarse ata o fin da transacción

```
CREATE TABLE EMP(  
  EMPNO NUMERIC(3) NOT NULL,  
  ENAME CHAR(20) NOT NULL,  
  EMAIL CHAR(25) NOT NULL CONSTRAINT U_EMAIL UNIQUE,  
  MGR NUMERIC(3), DEPTNO NUMERIC(3),  
  SAL NUMERIC(7,2),  
  CONSTRAINT PK_EMP PRIMARY KEY(EMPNO),  
  CONSTRAINT FK_EMP FOREIGN KEY(MGR)  
    REFERENCES EMP(EMPNO) ON DELETE CASCADE ON UPDATE CASCADE,  
  CONSTRAINT C_SAL_POS CHECK(SAL > 0) DEFERRABLE INITIALLY DEFERRED);
```

Introducción

Actividade sen triggers

Restriccións a nivel de BD: Assertions

- `CREATE ASSERTION <nome> CHECK <condicion>`

```
CREATE ASSERTION as_salarios_similares
CHECK( ( (SELECT MAX(SAL) FROM EMP)-
        (SELECT MIN(SAL) FROM EMP) ) <100);
```

```
CREATE ASSERTION deptos_pequenos
CHECK (NOT EXISTS
        (SELECT DEPTNO FROM EMP
         GROUP BY DEPTNO
         HAVING COUNT(*)>10));
```

Limitacións

- A reacción sempre é abortar
- Os SXBD non as implementan

Triggers

Regras ECA

Triggers

Programación baseada en eventos.

Tamén chamados regras ECA (**E**vento-**C**ondición-**A**cción).

- Cando se produce o evento, o trigger *dispárase*.
- A condición é opcional (se non existe, suponse que é certa).
- Se a condición é certa, execútase a acción.

Exemplos de uso

1. Control de salario: o salario debe ser positivo.
2. Control de NIF inválido.
3. Un departamento non pode ter máis de 10 empregados.
4. Un empregado non pode ganar máis ca o seu xefe.
5. Creación de claves surrogadas ($\approx$ campo autoincremental).
6. Actualiza o total dunha factura cando se engaden/borran/modifican liñas.
7. Gravar log de todos os accesos que modifican datos na táboa de empregados
8. Manda un correo ó suministrador cando o stock dun artigo baixa do límite.

Definición de reglas ECA para algún dos exemplos previos

A este nivel especificamos que facer, non como o faremos.

- (1) Control de salario inválido
 - *E*: insert; *C*: salario $\leq$ 0; *A*: Non permitir a inserción
 - *E*: update; *C*: salario $\leq$ 0; *A*: Non permitir a actualización
- (5) Creación de clave surrogada (campo id)
 - *E*: insert; *C*: —; *A*: Xenerar novo id
- (7) Log de modificacións na táboa de empregados
 - *E*: insert; *C*: —; *A*: Gravar entrada no log
 - *E*: delete; *C*: —; *A*: Gravar entrada no log
 - *E*: update; *C*: —; *A*: Gravar entrada no log

Triggers

A proposta do estándar SQL

Triggers no estándar SQL

- Dentro do estándar ISO/IEC 9075 Part 2: (SQL/Foundation)
- Especificados no SQL:1999 (antes SQL-3)
- Algunhas ampliacións en SQL:2003-SQL:2008
- Un trigger “controla” unha única táboa (ou vista)
- Eventos: insert, delete, update
- Un trigger responde a un único evento
- Un trigger non valida os datos existentes (como fan as restricións salvo se usamos novalidate)

Ventaxas dos triggers

- Simplifica a codificación de aplicacións de acceso a BDs
- Maior consistencia (control centralizado)

Triggers en SQL:2008

Elementos dun trigger

- Nome do trigger
- Nome da táboa
- **Evento:** insert, delete, update [OF <lista columnas>].
- Tempo de activación:
 - Antes (before) ou despois (after) do evento.
 - En vez de (instead of): *Non core, "Feature T213"*. Só aplicable a vistas.
- Granularidade: for each {row | statement}
Cantas veces se dispara o trigger para un evento:
Implica distintas visións dos datos: táboas e variables de transición (próxima transparencia)
 - A nivel sentencia (statement): unha vez para todo o evento
 - A nivel fila (row): unha vez para cada fila afectada
- **Condición:** when (<condición>). Pode omitirse.
- **Acción:**
 - Debe ser atómica.
 - O seguinte non se recomenda (aberto a implementación):
 - DML de modificación de datos en triggers before
(Podemos usar asignacións a variables de transición con set).
 - Sentencias de control de transaccións, conexión a BD, ou DDL.

Triggers en SQL:2008

Elementos dun trigger (ii)

Táboas e variables de transición

- Valores antigos (old) e novos (new) visibles para o trigger.
- A veces chamados “nomes de correlación”.
- Se son usados, é obrigatorio declaralos na cláusula referencing
- Variables de transición: para triggers for each row
 - Valores da fila previo e posterior á execución do evento.
 - Nomenclatura: old [row], new [row]
 - Non ambos valores son aplicables en todos os eventos (ex: old row para insert)
- Táboa de transición: para triggers for each statement
 - Valores previo e posterior das filas afectadas polo evento.
 - Nomenclatura: old table, new table
 - Non se refire á táboa completa (nomenclatura enganosa)!, só ó conxunto de filas afectadas (resaltadas no exemplo)
 - Non completamente implementado en SXBD reais
(o máis habitual é que vexan só a táboa completa antes/despois do evento, dependendo do tempo de activación).

EMP antes			⇒	EMP despois		
EMPNO	ENAME	SAL		EMPNO	ENAME	SAL
0001	JONES	1500	UPDATE EMP SET SAL=3100 WHERE SAL>1540	0001	JONES	1500
0002	SMITH	1600		0002	SMITH	3100
0010	QUEEN	1550		0010	QUEEN	3100
1000	KING	3000		1000	KING	3100

Triggers en SQL:2008

Sintaxe básica

CREATE TRIGGER <nome_trigger>

```
{BEFORE |  
  AFTER |  
  INSTEAD OF }{INSERT |  
    DELETE |  
    UPDATE [OF <columnas>] } ON <táboa>(*)
```

Evento

```
[REFERENCING OLD [ROW] [AS] <fila_vella> |  
  NEW [ROW] [AS] <fila_nova> |  
  OLD TABLE [AS] <táboa_vella> |  
  NEW TABLE [AS] <táboa_nova> ]
```

► Nomes de Correlación

```
[{ FOR EACH ROW | FOR EACH STATEMENT }]
```

► Granularidade

```
[ WHEN ( <condición> ) ]
```

Condición

```
{< Sentencia SQL> | BEGIN ATOMIC  
  <Sentencia SQL 1>;  
  ...  
  <Sentencia SQL N>  
  END }
```

Acción

(*) En triggers INSTEAD OF debe ser unha vista

Para borrar un trigger: DROP TRIGGER <nome_trigger>

Triggers en SQL:2008

Exemplos

Exemplo 1: Control de salario

- Usamos a táboa de exemplo de oracle:

```
emp(empno, ename, job, mgr, hiredate, sal, comm, deptno))
```

- Podería resolverse cunha restricción ou unha aserción:

```
ALTER TABLE emp
```

```
ADD CONSTRAINT sal_pos CHECK (sal>0)
```

```
CREATE ASSERTION emp_sal_pos
```

```
CHECK (NOT EXISTS (SELECT * FROM emp WHERE sal<=0))
```

- Comentarios

- Funcionan tanto para insert como para update.
- Comproban os datos existentes na BD.
- Poderían facerse aplazables ata o fin da transacción.
- Provocan erro e a inserción ou actualización non se leva a cabo.

Triggers en SQL:2008

Exemplos

Exemplo 1: control de salario (cont.)

Usando triggers

- Para a inserción, podemos eliminar o dato insertado (de forma transparente ó usuario), ou xenerar unha excepción.

```
CREATE TRIGGER t_emp_salpos_air
    AFTER INSERT ON Emp                -- Debe ser AFTER
    REFERENCING NEW ROW AS nova
    FOR EACH ROW
    WHEN (nova.sal <=0)
    DELETE FROM Emp
        WHERE empno = nova.empno;

CREATE TRIGGER t_emp_salpos_bir
    BEFORE INSERT ON Emp              -- Serviría BEFORE ou AFTER
    REFERENCING NEW ROW AS nova
    FOR EACH ROW
    WHEN (nova.sal <=0)
    SIGNAL SQLSTATE '99001', 'Salario inválido';
```

Triggers en SQL:2008

Exemplos

Exemplo 1: control de salario (cont.)

Para a actualización, deixando o salario anterior: Facer un novo update?

```
CREATE TRIGGER t_emp_salpos_aur
  AFTER UPDATE OF sal ON Emp          -- Debe ser AFTER
  REFERENCING NEW ROW AS nova
    OLD ROW AS vella
  FOR EACH ROW
  WHEN (nova.sal <=0)
  UPDATE Emp SET sal = vella.sal      -- Dispara o trigger de novo
    WHERE empno = nova.empno;        -- Menos eficiente
```

Mellor cambiar o valor na fila actual utilizando set (só válido nos triggers before).

```
CREATE TRIGGER t_emp_salpos_bur
  BEFORE UPDATE OF sal ON Emp        -- Debe ser BEFORE
  REFERENCING NEW ROW AS nova
    OLD ROW AS vella
  FOR EACH ROW
  WHEN (nova.sal <=0)                -- "Intercepta" o valor e substitúeo
  SET nova.sal = vella.sal;          -- polo anterior antes de gravar
```

Triggers en SQL:2008

Exemplos

Exemplo 5: Xeneración de id subrogado

- Usamos unha secuencia
- Creamos un trigger que, no caso da inserción, substitúe o valor do identificador polo próximo valor da secuencia.
- Ó modificar (con set) o valor nova, debe ser de tipo before.
-- Táboa: t1(id primary key, ...)

```
CREATE SEQUENCE seq_t1;
```

```
CREATE TRIGGER t_t1_id_bir
  BEFORE INSERT ON t1                -- Debe ser BEFORE
  REFERENCING NEW ROW AS nova
  FOR EACH ROW
  SET nova.id = NEXT VALUE FOR seq_t1;
```

Triggers en SQL:2008

Exemplos

Exemplo 7: Log de modificacións

- Trigger a nivel de sentencia: rexistra cada sentencia de modificación sobre EMP.
- Usamos unha táboa `log(id, usuario, evento, timestamp)`
- Usaremos unha secuencia `seq_log` para o identificador
- Poderíamos incluso almacenar no log os valores vellos/novos das modificacións (requeriría trigger a nivel de fila).
- Como non afecta á propia táboa, pode ser `before` ou `after`.

-- Móstrase só o trigger para insert.
-- Para delete e update serían análogos.

```
CREATE TRIGGER t_emp_log_ais
  AFTER INSERT ON EMP                -- Serviría BEFORE ou AFTER
  FOR EACH STATEMENT
  INSERT INTO log(id, usuario, evento, timestamp)
    VALUES(NEXT VALUE FOR seq_log,
            CURRENT_USER,
            'insert',
            CURRENT_TIMESTAMP);
```

Triggers en SQL:2008

Triggers INSTEAD OF

- Triggers utilizados só con vistas (non táboas).
- Non se permite a condición (cláusula when).
- Execútase a acción do trigger **en vez de** executar o evento.
- Utilidade principal: Facer actualizable unha vista que non o é.

```
create view vemp1 as
  select deptno, count(*) as numemps
    from emp
   group by deptno;
```

```
delete from vemp1          -- Falla!! Vista non actualizable.
  where deptno=10;
```

```
CREATE TRIGGER t_vemp1_borrado_iodr
  INSTEAD OF DELETE ON vemp1
  REFERENCING OLD ROW AS vella
  FOR EACH ROW
  DELETE FROM emp WHERE deptno = vella.deptno;
```

```
delete from vemp1          -- Funciona! Borra todos os empregados do depto 10.
  where deptno=10;
```


Triggers en Oracle

Comparación co estándar SQL:2008

Evento

- Inclúe máis tipos de evento que os DML do estándar:
Eventos DDL (`create on schema, ...`), base de datos (`logon, servererror, ...`), etc.
- Permite controlar máis dun evento (usando `or`)
 - Deben coincidir no tempo de activación e granularidade.
 - Uso de predicados `inserting`, `updating` e `deleting` na acción.
- Incluía triggers `instead of` antes de estar presente no estándar.
- Táboas e variables de transición:
 - Non usa táboas de transición (`old/new table`)
 - Si que usa variables de transición (`old/new`) (non admite a palabra `row`).
Só triggers de fila. Nomes por defecto: `old` e `new`
`old` nunca pode ser modificado; `new` só nos triggers `before`
Cando non son aplicables (ex: `old` en trigger de `insert`) conteñen `NULL`

Condición

- Limitada: non admite subconsultas. Pode omitirse a cláusula `when` e usar `if` na acción.

Acción

- Bloque PL/SQL ou chamada a un procedemento.
- Non se permiten sentencias de control transaccional (`commit, rollback, ...`)
- Na acción, as variables de transición van precedidas do símbolo dous puntos (`:new, :old`).

Triggers en Oracle

Creación de triggers

- Para crear ou modificar un trigger

```
CREATE [OR REPLACE] TRIGGER <nome_trigger>
  {BEFORE|AFTER|INSTEAD OF}
      <Evento> [OR <Evento>...] ON <Tabla>
  [ REFERENCING [NEW AS tupla_nova]
                [OLD AS tupla_vella] ]
  [FOR EACH ROW]
  [WHEN (<Condicion_SQL_simple>)]
{ <Bloque PL/SQL> | CALL <procedemento(argumentos)> }
```

- Para borrar: DROP TRIGGER <nome_trigger>

- Para activar ou desactivar:

- Triggers individuais

```
ALTER TRIGGER <nome_trigger> {ENABLE|DISABLE}
```

- Todos os triggers asociados a unha táboa

```
ALTER TABLE <táboa> {ENABLE|DISABLE} ALL TRIGGERS
```

Triggers en Oracle

Datos no catálogo

- Para consultar os existentes:

Táboas ALL_TRIGGERS, USER_TRIGGERS

```
SQL> desc user_triggers
```

Nombre	¿Nulo?	Tipo
TRIGGER_NAME		VARCHAR2(30)
TRIGGER_TYPE		VARCHAR2(16)
TRIGGERING_EVENT		VARCHAR2(227)
TABLE_OWNER		VARCHAR2(30)
BASE_OBJECT_TYPE		VARCHAR2(16)
TABLE_NAME		VARCHAR2(30)
COLUMN_NAME		VARCHAR2(4000)
REFERENCING_NAMES		VARCHAR2(128)
WHEN_CLAUSE		VARCHAR2(4000)
STATUS		VARCHAR2(8)
DESCRIPTION		VARCHAR2(4000)
ACTION_TYPE		VARCHAR2(11)
TRIGGER_BODY		LONG

```
select trigger_name, trigger_type, triggering_event, when_clause, status
  from user_triggers
 where table_name='EMP';
```

TRIGGER_NAME	TRIGGER_TYPE	TRIGGER	WHEN_CLAUSE	STATUS
T_EMP_SALPOS_BUR	BEFORE EACH ROW	UPDATE	nova.SAL<=0	ENABLED

```
1 fila seleccionada.
```

Triggers en Oracle

Exemplos

Exemplo 1: control de salario

- A opción de xenerar unha excepción funciona similar ó estándar.
- Omitimos a cláusula referencinng e usamos new.
- Insertando un salario negativo vemos o código e mensaxe de erro (e non se inserta a fila).

```
CREATE OR REPLACE TRIGGER t_emp_salpos_bir
  BEFORE INSERT ON Emp
  FOR EACH ROW
  WHEN (new.sal <=0)
  BEGIN
    RAISE_APPLICATION_ERROR(-20001,'Salario '||:new.sal||' inválido');
  END;
/
```

Disparador creado

```
insert into emp (empno, sal) values (1234, -2500)
*
```

ERROR en línea 1:

ORA-20001: Salario -2500 inválido

ORA-06512: en "PENABAD.T_EMP_SALPOS_BIR", línea 2

ORA-04088: error durante la ejecución del disparador 'PENABAD.T_EMP_SALPOS_BIR'

Triggers en Oracle

Exemplos

Exemplo 1: control de salario (cont.)

- A opción de eliminar a fila insertada non funciona. O trigger créase correctamente...

```
CREATE OR REPLACE TRIGGER t_emp_salpos_air
  AFTER INSERT ON Emp
  FOR EACH ROW
  WHEN (new.sal <=0)
  BEGIN
    DELETE FROM Emp
      WHERE empno = :new.empno;
  END;
```

- Pero o comportamento non é o esperado

```
insert into emp (empno, sal) values (1234, -2500)
*
```

ERROR en línea 1:
ORA-04091: la tabla PENABAD.EMP está mutando, puede que el disparador/la función no puedan verla
ORA-06512: en "PENABAD.T_EMP_SALPOS_AIR", línea 2
ORA-04088: error durante la ejecución del disparador 'PENABAD.T_EMP_SALPOS_AIR'

- A situación conócese como *táboa mutante*. En xeral, nun trigger a nivel de fila non se pode acceder con SQL á táboa controlada polo trigger (nin modificacións nin select).

Triggers en Oracle

Exemplos

Exemplo 1: control de salario (cont.)

- Non se usa a opción de facer un segundo update para deixar o salario anterior (non era recomendable, e ademais provocaría a táboa mutante).
- Si poderíamos xenerar unha excepción.
- Neste caso, *interceptamos* o salario novo e sobreescribímolo co antigo. Como vemos, anque aparentemente se actualiza unha fila, o valor non cambia.

```
CREATE OR REPLACE TRIGGER t_emp_salpos_bur
  BEFORE UPDATE ON Emp
  FOR EACH ROW
  WHEN (new.sal <= 0)
BEGIN
  :new.sal := :old.sal;
END;
```

```
select empno, ename, sal from emp where empno=7369;
7369 SMITH          800
update emp set sal=-1 where empno=7369;
1 fila actualizada.
select empno, ename, sal from emp where empno=7369;
7369 SMITH          800
```

Triggers en Oracle

Exemplos

Exemplo 1: control de salario (cont.)

- Podemos mezclar ambos triggers nun só (coincide granularidade e tempo de activación)
- Usamos o predicado `updating` para saber se o evento disparador foi `update`.

```
CREATE OR REPLACE TRIGGER t_emp_salpos_biur
  BEFORE INSERT OR UPDATE ON Emp
  FOR EACH ROW
  WHEN (new.sal <= 0)
BEGIN
  IF updating THEN
    :new.sal := :old.sal;
  ELSE -- inserting
    RAISE_APPLICATION_ERROR(-20001,'Salario '||:new.sal||' inválido');
  END IF;
END;
```

Triggers en Oracle

Exemplos

Exemplo 5: Xeneración de id subrogado

- Similar a SQL estándar (só diferencias sintácticas).
- Recordad: as secuencias de SQL non garantizan números secuenciais, só distintos.

```
CREATE TABLE T1(  
    ID NUMBER(3) CONSTRAINT PK_T1 PRIMARY KEY,  
    A VARCHAR(5));  
CREATE SEQUENCE seq_t1;  
  
CREATE TRIGGER t_t1_id_bir  
    BEFORE INSERT ON t1  
    REFERENCING NEW AS nova    -- Poderíamos omitilo e usar new  
    FOR EACH ROW  
    BEGIN  
        :nova.id := seq_t1.nextval;  
    END;  
  
insert into t1(a) values ('fila1');  
insert into t1(a) values ('fila2');  
select * from t1;  
    1 fila1  
    2 fila2
```


Triggers en Oracle

Exemplos

Exemplo 7: Log de modificacións

- En Oracle podemos crear un trigger para os 3 eventos

-- A nivel de sentencia (omitindo FOR EACH ROW): Inserta unha fila en "log" por
-- cada operación DML sobre emp, independentemente do número de filas afectadas.

```
CREATE OR REPLACE TRIGGER t_emp_log_air
  AFTER INSERT OR UPDATE OR DELETE ON EMP    -- Serviría BEFORE ou AFTER
  DECLARE op VARCHAR2(20);
BEGIN
  IF INSERTING THEN      op := 'insert';
  ELSIF UPDATING THEN    op := 'update';
  ELSE /* DELETING */    op := 'delete';
  END IF;

  INSERT INTO log(id, usuario, evento, timestamp)
    VALUES(seq_log.nextval, USER, op, CURRENT_TIMESTAMP);
END;

insert into emp(empno, ename) values (1234, 'Novo');
  1 fila creada.
select * from log;
  1 PENABAD insert 29/01/20 11:10:02,644781
```

Triggers en Oracle

Triggers instead of

Similar a SQL estándar (só diferencias sintácticas).

Fai vistas actualizables cando non o eran, ou cambia a forma de actualizar nas que xa eran actualizables (permite engadir controis, etc.).

Exemplos

```
create view vemp1 as
  select deptno, count(*) as numemps
    from emp
   group by deptno;
```

```
delete from vemp1 where deptno=10;
ERROR en línea 1:
ORA-01732: operación de manipulación de datos no válida en esta vista
```

```
CREATE OR REPLACE TRIGGER t_vemp1_borrado_iodr
  INSTEAD OF DELETE ON vemp1
  FOR EACH ROW
  BEGIN
    DELETE FROM emp WHERE deptno = :old.deptno;
  END;
```

```
delete from vemp1 where deptno=10; -- Realmente suprime 3 filas en emp
1 filas suprimida.
```

Triggers en Oracle

Triggers instead of

Exemplos (cont.)

```
create view empdep as
  select * from emp natural join dept;

-- Non permite a inserción: vista con join non actualizable
insert into empdep
  values(10, 7001, 'PARKER', 'CLERK', 7782, '12/01/2015',
        1800, NULL, 'VENTAS', 'LONDON')
ERROR en línea 1:
ORA-01779: no se puede modificar una columna que se corresponde con
una tabla no reservada por clave

-- Nota: Se a vista fose con equijoin en vez de join natural, Oracle
-- permite a modificación das columnas da "táboa preservada por clave",
-- neste caso EMP.

-- Decidimos unha forma manual de facer a actualización.
-- Por exemplo:
-- 1. Se o departamento existe, actualízase tanto o nome como a localidade.
--    Se non existe, créase
-- 2. Se o empregado existe, xenérase unha excepción.
--    Se non existe, insértase.
```

Triggers en Oracle

Triggers instead of

Exemplos (cont.)

```
CREATE OR REPLACE TRIGGER t_empdep_insert_ioir
INSTEAD OF INSERT ON empdep
FOR EACH ROW
DECLARE n NUMBER;
BEGIN
    SELECT count(*) INTO n FROM emp
        WHERE empno = :new.empno;
    IF n>0 THEN
        RAISE_APPLICATION_ERROR(-20036, 'Empleado existente');
    END IF;
    UPDATE dept SET dname=:new.dname, loc=:new.loc
        WHERE deptno=:new.deptno;
    IF SQL%NOTFOUND THEN -- Non existía: inserto
        INSERT INTO dept(deptno, dname, loc) values(:new.deptno, :new.dname, :new.loc);
    END IF;
    INSERT INTO emp(empno, ename, job, mgr, hiredate, sal, comm, deptno)
        values(:new.empno, :new.ename, :new.job, :new.mgr,
            :new.hiredate, :new.sal, :new.comm, :new.deptno);
END;
```

Triggers en Oracle

Triggers instead of

Exemplos (cont.)

-- Proba de funcionamento:

```
select * from dept where deptno = 10;  
10 ACCOUNTING      NEW YORK
```

```
select * from emp where empno = 7001;  
-- Non hai filas
```

```
insert into empdep  
values(10, 7001, 'PARKER', 'CLERK', 7782, '12/01/2015',  
1800, NULL, 'VENTAS', 'LONDON');  
1 fila creada
```

```
select * from dept where deptno = 10;  
10 VENTAS          LONDON
```

```
select * from emp where empno = 7001;  
7001 PARKER        CLERK        7782 12/01/15    1,800          10
```

Bibliografía

- [1] Ramez A. Elmasri e Shamkant B. Navathe. *Fundamentos de sistemas de bases de datos*. 5ª ed. Addison Wesley, 2007.
- [2] J. Melton e A. R. Simon. *SQL:1999 - Understanding relational language components*. Morgan Kaufmann, 2002. ISBN: 1558604561.
- [3] *Oracle Database 19c*. <https://docs.oracle.com/en/database/oracle/oracle-database/19>. 2023.
- [4] P. Gultzan e T. Pelzer. *SQL-99 complete, really*. Kansas: R&D Books, 1999. ISBN: 0879305681.
- [5] *PEP 249 – Python Database API Specification v2.0*.
<https://www.python.org/dev/peps/pep-0249/>.
- [6] *PostgreSQL 13 documentation*. <https://www.postgresql.org/docs/13/>.
- [7] *Psycopg – PostgreSQL database adapter for Python*. <https://www.psycopg.org/docs/>.
- [8] Abraham Silberschatz, Henry F. Korth e S. Sudarshan. *Database System Concepts*. 6ª. McGraw-Hill, 2011.